# Guía de implementación completa

Sistema de registro de piezas con login por roles (trabajador / ejecutivo), hospedado en GitHub Pages con Firebase como backend. Incluye todos los pasos: Firebase, cada archivo del proyecto con su código completo, pruebas locales y publicación con GitHub Desktop.

---

**Contenido**


1 Qué necesitas antes de empezar
2 Qué hace este sistema y cómo se guardan los datos
3 Crear el proyecto en Firebase y registrar la app web
4 Crear los usuarios (trabajador y ejecutivo)
5 Pegar las reglas de seguridad de Firestore
6 Crear la carpeta del proyecto
7 Armar cada archivo, paso a paso (con código completo)
8 Cómo se conecta todo: el <script type="module">
9 Probar el sitio en tu computadora
10 Subir el proyecto con GitHub Desktop y activar GitHub Pages
11 Alerta "Secrets detected" de GitHub y proteger la API key (opcional)
12 Probar el sitio publicado
13 Agregar un usuario nuevo más adelante
14 Cambios comunes (zonas, colores, campos nuevos)
15 Siguientes pasos posibles


## 1. Qué necesitas antes de empezar

- Una **cuenta de Google** (para Firebase).
- Una **cuenta de GitHub** y la app **GitHub Desktop** instalada (desktop.github.com).
- **Visual Studio Code** (code.visualstudio.com) para crear y editar los archivos.
- En VS Code, la extensión **Live Server** (de Ritwick Dey): ábrela desde el ícono de Extensiones (cuadritos en la barra izquierda), búscala por nombre e instálala. Sirve para probar el sitio en tu computadora (paso 9).

## 2. Qué hace este sistema y cómo se guardan los datos

Es un sitio web con tres pantallas: una de **inicio de sesión**, un **formulario** para que el trabajador registre el movimiento de una pieza, y un **dashboard** para que el ejecutivo consulte y filtre todos los registros. Los datos viven en **Firestore** (la base de datos de Firebase) y el sitio se publica gratis en **GitHub Pages**.

Flujo de uso:

1. La persona entra con correo y contraseña (Firebase Authentication).
2. El sistema busca su perfil en la colección **usuarios** y lee su campo **rol**.
3. Si es **trabajador** lo manda al formulario; si es **ejecutivo**, al dashboard.
4. El trabajador escribe primero el **folio**, que es el identificador de la pieza. Si ese folio ya existe, el sistema **autollena** nombre, cliente, material y cantidad, y pone como zona de origen la última zona a donde se movió la pieza. Si no existe, avisa que es un folio nuevo y se llena a mano.
5. Los datos autollenados se pueden corregir antes de registrar.
6. Al registrar se guardan dos cosas a la vez: el movimiento (un documento nuevo en **registros**) y la ficha de la pieza (en **piezas**, se crea si es nueva o se actualiza con lo corregido). Después el formulario se limpia para capturar el siguiente.

Datos que se guardan en Firestore:

| Colección | Campo | Contenido |
|---|---|---|
| usuarios | (ID del documento) | El UID del usuario en Authentication |
| usuarios | nombre | Nombre de la persona |
| usuarios | rol | "trabajador" o "ejecutivo" |
| piezas | (ID del documento) | El folio de la pieza (en mayúsculas) |
| piezas | folio / nombre | Folio y nombre de la pieza |
| piezas | cliente / material / cantidad | Datos de la pieza (opcionales) |
| piezas | ultimaZona | Última zona destino a la que se movió |
| piezas | actualizadoPor / actualizadoEn | Quién y cuándo la modificó por última vez |
| registros | folio | Folio de la pieza |
| registros | nombrePieza | Nombre de la pieza al momento del movimiento |
| registros | cliente / material / cantidad | Datos de la pieza al momento del movimiento |
| registros | zonaOrigen | Una de la lista ZONAS_ORIGEN (js/zonas.js) |
| registros | zonaDestino | Una de la lista ZONAS_DESTINO (js/zonas.js) |
| registros | comentarios | Texto libre (opcional) |
| registros | nombreTrabajador / uidTrabajador | Quién hizo el registro (se llena solo) |
| registros | creadoEn | Fecha y hora del servidor (se llena solo) |

## 3. Crear el proyecto en Firebase y registrar la app web

1. Entra a **console.firebase.google.com** con tu cuenta de Google.
2. Clic en **"Agregar proyecto"**, ponle un nombre (ej. RegistroPiezas) y crea el proyecto.

3. Al terminar, Firebase te deja en la pantalla de bienvenida del proyecto ("Hola [tu nombre]", con tarjetas sugeridas de Gemini). Ignora esas tarjetas y busca el botón **"+ Agregar app"**, arriba junto al nombre del proyecto.
4. Firebase muestra varios íconos circulares de plataforma en este orden: iOS, Android, Web, Unity y Flutter. **No elijas Android** (aunque sea el que aparece resaltado al pasar el mouse) ni iOS. El ícono correcto es el tercero, el que tiene el símbolo **</>** — ese es "Web".
5. Dale un apodo a la app (ej. "RegistroPiezas Web"). **No** marques la casilla de Firebase Hosting, ya que el sitio se va a publicar en GitHub Pages.
6. Clic en **"Registrar app"**.
7. Firebase te lleva a la pantalla **"Agrega el SDK de Firebase"**, con dos opciones: "Usar npm" o "Usar una etiqueta <script>". Elige **"Usar una etiqueta <script>"**: este proyecto no usa npm ni un empaquetador (Webpack/Rollup), carga Firebase directo desde internet (CDN).
8. De esa pantalla solo necesitas dos cosas: el bloque **const firebaseConfig = { ... }** (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId) y el **número de versión** que aparece en la dirección del import (ej. `firebasejs/13.1.0/`). Cópialos — los usas en el paso 7.
9. Clic en "Ir a la consola" para cerrar el asistente.
10. En el menú lateral entra a **Seguridad** y, dentro de ese submenú, clic en **Authentication**. Da clic en "Comenzar".
11. Dentro de Authentication, pestaña "Sign-in method", activa el proveedor **Correo electrónico/contraseña**.
12. En el menú lateral entra a **Bases de datos y almacenamiento**. Se abre un submenú con tres grupos: PostgreSQL (SQL Connect), NoSQL (Firestore, Realtime Database) y Almacenamiento de objetos (Storage). Dentro del grupo **NoSQL**, clic en **Firestore**. Da clic en "Crear base de datos", elige **modo de producción** y la región más cercana (ej. us-central).

> "Modo de producción" significa que la base empieza bloqueada: nadie puede leer ni escribir hasta que pegues las reglas del paso 5. Si más adelante necesitas volver a ver el firebaseConfig, está en el ícono de engrane → **Configuración del proyecto**, sección "Tus apps".

## 4. Crear los usuarios (trabajador y ejecutivo)

Por ahora los usuarios se crean a mano desde la consola:

1. En **Seguridad** → **Authentication** → **Users**, clic en "Agregar usuario". Captura un correo y una contraseña (mínimo 6 caracteres) para cada persona.
2. Copia el **UID** que Firebase asigna a cada usuario (aparece en la lista de usuarios).
3. Ve a **Firestore** → **Datos** y clic en "Iniciar colección". Como ID de la colección escribe **usuarios** (en minúsculas).
4. Firebase te muestra un **ID de documento** generado automáticamente (algo como "61b7lcJw8HjIKAaeUYUg"). **Bórralo y pega ahí el UID** que copiaste — tiene que quedar idéntico, porque así el sistema relaciona la cuenta de login con su rol.

5 Agrega dos campos, ambos de tipo **string**:
   • **nombre** = el nombre de la persona (ej. "Ana")
   • **rol** = exactamente **trabajador** o **ejecutivo** (en minúsculas). Para el segundo campo usa el botón **"Agregar campo"**.

6 Clic en **Guardar**.

7 Para los siguientes usuarios, dentro de la colección **usuarios** usa "Agregar documento" y repite lo mismo (un documento por cada UID).

**Así debe verse un usuario bien creado:**

```
usuarios  >  cQi478uMQnWAD8QVR17gWC1HWW13
              nombre: "Ana"
              rol:    "ejecutivo"
```

Crea al menos un usuario de cada rol para poder probar las dos pantallas.

## 5. Pegar las reglas de seguridad de Firestore

Estas reglas son las que realmente protegen los datos. Sin ellas, la base sigue bloqueada.

1 En Firestore, ve a la pestaña **Reglas**.

2 Borra todo el contenido y pega el código de abajo (es el mismo que el archivo **firestore.rules** del proyecto, paso 7.12).

3 Clic en **Publicar**.

- Cualquier usuario con sesión puede **crear** registros.
- Solo el rol **ejecutivo** puede **leer/listar** todos los registros.
- Cualquier usuario con sesión puede **leer** la ficha de una pieza (para el autollenado) y **crearla o actualizarla** al registrar un movimiento; nadie puede borrar fichas.
- Nadie puede editar ni borrar registros ya creados.
- Cada usuario solo puede leer su propio perfil; los perfiles solo se crean desde la consola.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function estaAutenticado() {
      return request.auth != null;
    }

    function rolDeUsuario() {
      return get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data.rol;
    }

    // Solo cada usuario puede leer su propio perfil. Se crean a mano desde la consola.
    match /usuarios/{userId} {
      allow read: if estaAutenticado() && request.auth.uid == userId;
      allow write: if false;
    }

    // Cualquier usuario con sesión puede crear registros (trabajador).
    // Solo el rol "ejecutivo" puede leer/listar todos los registros.
    // Nadie puede editar ni borrar registros ya creados.
    // Ficha de cada pieza, con el folio como ID. Todos los usuarios con sesión la pueden leer
    // (para el autollenado) y crear/actualizar al registrar un movimiento. Nadie la puede borrar.
    match /piezas/{folio} {
      allow read: if estaAutenticado();
```

```
      allow create, update: if estaAutenticado() && request.resource.data.folio == folio;
      allow delete: if false;
    }

    match /registros/{registroId} {
      allow create: if estaAutenticado();
      allow read, list: if estaAutenticado() && rolDeUsuario() == 'ejecutivo';
      allow update, delete: if false;
    }
  }
}
```

## 6. Crear la carpeta del proyecto

1. Crea una carpeta en tu computadora, ej. **RegistroPiezas**.
2. Abre VS Code → **File** → **Open Folder...** y elige esa carpeta.
3. Dentro crea dos subcarpetas: **css** y **js** (en el panel izquierdo de VS Code, ícono de "New Folder").
4. Para cada archivo del paso 7: clic derecho sobre la carpeta correcta → **New File**, escribe el nombre exacto (con su extensión), pega el código y guarda con **Ctrl+S**.

**Así debe quedar la estructura al terminar:**

```
RegistroPiezas/
|-- index.html              <- pantalla de login
|-- registro.html           <- formulario del trabajador
|-- dashboard.html          <- panel del ejecutivo
|-- firestore.rules         <- copia de las reglas de seguridad
|-- css/
|   `-- styles.css          <- estilos de todo el sitio
`-- js/
    |-- firebase-config.js  <- tus datos de Firebase
    |-- firebase-init.js    <- conexión con Firebase
    |-- auth-guard.js       <- protege páginas por rol
    |-- zonas.js            <- lista de zonas (se editan aquí)
    |-- login.js            <- lógica del login
    |-- registro.js         <- lógica del formulario
    `-- dashboard.js        <- lógica del dashboard
```

> Los nombres de archivos y carpetas deben ser exactamente estos (minúsculas, sin espacios), porque los archivos se llaman entre sí por nombre.

# 7. Armar cada archivo, paso a paso

El orden recomendado es: primero los estilos y la conexión con Firebase y la lista de zonas (7.1 a 7.5), luego cada pantalla con su HTML y su JS (7.6 a 7.11), y al final la copia de las reglas (7.12).

## 7.1 styles.css

**Ruta: css/styles.css**

Los estilos visuales de las tres pantallas. Es el único archivo CSS del proyecto.

Qué hace por dentro:

- **:root** define los colores como variables (`--azul`, `--verde`, etc.). Para cambiar los colores del sitio solo editas ahí.
- **.topbar** es la barra azul de arriba con el nombre y el botón Salir.
- **.card** es la tarjeta blanca donde va cada formulario.
- **.mensaje.ok** / **.mensaje.error** son los avisos verde y rojo.
- **.filtros** y **table** dan forma al dashboard.

**Código completo:**

```
:root {
  --azul: #1d4ed8;
  --azul-oscuro: #1e3a8a;
  --gris-fondo: #f3f4f6;
  --gris-borde: #d1d5db;
  --texto: #111827;
  --verde: #16a34a;
  --rojo: #dc2626;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  background: var(--gris-fondo);
  color: var(--texto);
}

.topbar {
  background: var(--azul-oscuro);
  color: white;
  padding: 14px 20px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.topbar h1 {
  font-size: 18px;
  margin: 0;
}

.topbar button {
  background: transparent;
  border: 1px solid white;
  color: white;
  padding: 6px 12px;
  border-radius: 6px;
  cursor: pointer;
}
```

```
.topbar button:hover {
  background: rgba(255,255,255,0.15);
}

.contenedor {
  max-width: 720px;
  margin: 40px auto;
  padding: 0 16px;
}

.card {
  background: white;
  border-radius: 10px;
  padding: 28px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}

.card h2 {
  margin-top: 0;
}

label {
  display: block;
  font-weight: 600;
  margin: 14px 0 6px;
  font-size: 14px;
}

input, select, textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid var(--gris-borde);
  border-radius: 6px;
  font-size: 15px;
  font-family: inherit;
}

textarea { resize: vertical; min-height: 70px; }

button.primario {
  margin-top: 20px;
  width: 100%;
  background: var(--azul);
  color: white;
  border: none;
  padding: 12px;
  border-radius: 6px;
  font-size: 16px;
  cursor: pointer;
}

button.primario:hover { background: var(--azul-oscuro); }

.mensaje {
  margin-top: 14px;
  padding: 10px 12px;
  border-radius: 6px;
  font-size: 14px;
  display: none;
}

.mensaje.ok {
  display: block;
  background: #dcfce7;
  color: #166534;
  border: 1px solid #86efac;
}

.mensaje.error {
  display: block;
  background: #fee2e2;
```

```
  color: #991b1b;
  border: 1px solid #fca5a5;
}

.estado-folio {
  font-size: 13px;
  margin-top: 6px;
  min-height: 18px;
  color: #6b7280;
}

.estado-folio.encontrado { color: var(--verde); }
.estado-folio.nuevo { color: var(--azul); }

.fila-dos {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.dashboard-contenedor {
  max-width: 1100px;
  margin: 24px auto;
  padding: 0 16px;
}

.filtros {
  background: white;
  border-radius: 10px;
  padding: 18px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 12px;
  align-items: end;
  margin-bottom: 20px;
}

.filtros label { margin: 0 0 6px; }

.filtros button {
  background: var(--azul);
  color: white;
  border: none;
  padding: 10px;
  border-radius: 6px;
  cursor: pointer;
  height: 40px;
}

table {
  width: 100%;
  border-collapse: collapse;
  background: white;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}

th, td {
  padding: 10px 12px;
  text-align: left;
  border-bottom: 1px solid #eee;
  font-size: 14px;
}

th {
  background: #f9fafb;
  font-weight: 600;
}
```

```
tr:hover td { background: #f9fafb; }

.vacio {
  text-align: center;
  color: #6b7280;
  padding: 24px;
}

.cargando {
  text-align: center;
  color: #6b7280;
  padding: 20px;
}
```

## 7.2 firebase-config.js

**Ruta: js/firebase-config.js**

Guarda los datos que identifican a tu proyecto de Firebase. Es el **único archivo que cambia de un proyecto a otro**; todo lo demás se queda igual.

**ANTES — así se crea el archivo, con valores de ejemplo:**

```
// Reemplaza estos valores con los de tu proyecto de Firebase.
export const firebaseConfig = {
  apiKey: "TU_API_KEY",
  authDomain: "TU_PROYECTO.firebaseapp.com",
  projectId: "TU_PROYECTO",
  storageBucket: "TU_PROYECTO.firebasestorage.app",
  messagingSenderId: "TU_SENDER_ID",
  appId: "TU_APP_ID"
};
```

**DESPUÉS — reemplazando cada valor por el del firebaseConfig que copiaste en el paso 3:**

```
// Configuración del proyecto RegistroPiezas en Firebase.
// Si necesitas volver a verla: Firebase Console -> Configuración del proyecto (ícono de engrane) ->
General -> "Tus apps".
export const firebaseConfig = {
  apiKey: "AIzaSyDIkx3_L42IEhr3T9qbRblag_TFgp2Nw8s",
  authDomain: "registropiezas-6039d.firebaseapp.com",
  projectId: "registropiezas-6039d",
  storageBucket: "registropiezas-6039d.firebasestorage.app",
  messagingSenderId: "151928281711",
  appId: "1:151928281711:web:fde50855cb384be4d10b9a"
};
```

> Copia solo los valores entre comillas. Fíjate que el archivo empieza con `export const` y no solo `const` como en la pantalla de Firebase: el `export` es lo que permite que firebase-init.js lo use. El storageBucket ahora termina en ".firebasestorage.app" (antes era ".appspot.com"); usa el valor exacto que te dé tu consola.

## 7.3 firebase-init.js

**Ruta: js/firebase-init.js**

Hace la conexión con Firebase una sola vez y la comparte con el resto del sitio. Es la versión organizada del bloque <script> que te mostró Firebase en el paso 3.

Qué hace por dentro:

- Importa Firebase directo desde internet (`https://www.gstatic.com/firebasejs/13.1.0/...`). Ese 13.1.0 es la versión que mostró la consola; si la tuya muestra otra, usa la tuya en **todos** los archivos .js.
- Importa tu `firebaseConfig` del archivo anterior.

- `initializeApp` crea la conexión; `getAuth` prepara el login y `getFirestore` la base de datos.
- Exporta `auth` y `db` para que los demás archivos los usen.

Este archivo no se vuelve a tocar, salvo para actualizar el número de versión.

**Código completo:**

```
import { initializeApp } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-app.js";
import { getAuth } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.js";
import { getFirestore } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js";
import { firebaseConfig } from "./firebase-config.js";

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

## 7.4 auth-guard.js

**Ruta: js/auth-guard.js**

El "guardia" de las páginas privadas. registro.html y dashboard.html lo usan para que nadie entre sin sesión o con el rol equivocado (por ejemplo, un trabajador escribiendo dashboard.html en la barra del navegador).

Qué hace por dentro:

- `protegerPagina(rol)` espera a que Firebase diga si hay sesión (`onAuthStateChanged`).
- Si no hay sesión, regresa a index.html.
- Si hay sesión, lee el documento `usuarios/{uid}`. Si no existe, cierra la sesión; si el rol no coincide, manda a la persona a su pantalla correcta.
- Si todo está bien, entrega uid, nombre y rol a la página.
- `configurarBotonSalir` conecta el botón Salir con `signOut`.

**Código completo:**

```
import { onAuthStateChanged, signOut } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.
js";
import { doc, getDoc } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js";
import { auth, db } from "./firebase-init.js";

// Revisa que haya sesión y que el rol coincida con el permitido para la página actual.
// rolRequerido: "trabajador" o "ejecutivo"
// Devuelve una Promise que resuelve con { uid, nombre, rol } cuando todo es válido.
export function protegerPagina(rolRequerido) {
  return new Promise((resolve) => {
    onAuthStateChanged(auth, async (user) => {
      if (!user) {
        window.location.href = "index.html";
        return;
      }

      const usuarioDoc = await getDoc(doc(db, "usuarios", user.uid));
      if (!usuarioDoc.exists()) {
        await signOut(auth);
        window.location.href = "index.html";
        return;
      }

      const datos = usuarioDoc.data();
      if (datos.rol !== rolRequerido) {
        window.location.href = datos.rol === "ejecutivo" ? "dashboard.html" : "registro.html";
        return;
      }

      resolve({ uid: user.uid, nombre: datos.nombre || user.email, rol: datos.rol });
```

```
    });
  });
}

export function configurarBotonSalir(idBoton) {
  document.getElementById(idBoton).addEventListener("click", async () => {
    await signOut(auth);
    window.location.href = "index.html";
  });
}
```

## 7.5 zonas.js

**Ruta: js/zonas.js**

La lista de zonas de origen y destino. Es el **único lugar donde se editan las zonas**: el formulario y los filtros del dashboard toman la lista de aquí, así que siempre coinciden.

Qué hace por dentro:

- `ZONAS_ORIGEN` son las opciones de "Zona de origen" (incluye Externo).
- `ZONAS_DESTINO` son las opciones de "Zona destino" (sin Externo).
- Los nombres pueden ser cualquier texto ("Almacén", "Línea 3", "Embarques"...), no tienen que ser letras. El orden en la lista es el orden en que aparecen en pantalla.
- `llenarSelectZonas` agrega cada nombre como opción a un `<select>`; la usan registro.js y dashboard.js.

**Código completo:**

```
// Lista de zonas del sistema. Este es el ÚNICO lugar donde se editan.
// Puedes cambiar los nombres, quitar o agregar zonas: cada una va entre comillas y separada por coma.

export const ZONAS_ORIGEN = [
  "Externo",
  "Zona A", "Zona B", "Zona C", "Zona D", "Zona E", "Zona F", "Zona G",
  "Zona H", "Zona I", "Zona J", "Zona K", "Zona L", "Zona M", "Zona N",
  "Zona O", "Zona P", "Zona Q", "Zona R", "Zona S", "Zona T", "Zona U",
  "Zona V", "Zona W", "Zona X", "Zona Y", "Zona Z"
];

export const ZONAS_DESTINO = [
  "Zona A", "Zona B", "Zona C", "Zona D", "Zona E", "Zona F", "Zona G",
  "Zona H", "Zona I", "Zona J", "Zona K", "Zona L", "Zona M", "Zona N",
  "Zona O", "Zona P", "Zona Q", "Zona R", "Zona S", "Zona T", "Zona U",
  "Zona V", "Zona W", "Zona X", "Zona Y", "Zona Z"
];

export function llenarSelectZonas(idSelect, zonas) {
  const select = document.getElementById(idSelect);
  zonas.forEach(nombre => {
    const opcion = document.createElement("option");
    opcion.value = nombre;
    opcion.textContent = nombre;
    select.appendChild(opcion);
  });
}
```

## 7.6 index.html

**Ruta: index.html (en la raíz, no dentro de una carpeta)**

La pantalla de inicio de sesión. Se llama index.html porque es la que GitHub Pages abre por defecto al entrar a la dirección del sitio.

Qué hace por dentro:

- En el `<head>` carga los estilos con `<link rel="stylesheet" href="css/styles.css">`.
- El formulario tiene dos campos con `id="correo"` y `id="contrasena"`; esos id son los que busca login.js.
- El `<div id="mensaje">` está vacío: ahí aparece el error si la contraseña es incorrecta.
- Al final del `<body>` va `<script type="module" src="js/login.js">`, que carga la lógica.

**Código completo:**

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Iniciar sesión - Registro de Piezas</title>
<link rel="stylesheet" href="css/styles.css" />
</head>
<body>
  <div class="contenedor" style="margin-top:80px;">
    <div class="card">
      <h2>Iniciar sesión</h2>
      <form id="form-login">
        <label for="correo">Correo</label>
        <input type="email" id="correo" required autocomplete="username" />

        <label for="contrasena">Contraseña</label>
        <input type="password" id="contrasena" required autocomplete="current-password" />

        <button type="submit" class="primario">Entrar</button>
        <div id="mensaje" class="mensaje"></div>
      </form>
    </div>
  </div>

  <script type="module" src="js/login.js"></script>
</body>
</html>
```

## 7.7 login.js

**Ruta: js/login.js**

La lógica de la pantalla de login.

Qué hace por dentro:

- Cuando se envía el formulario, evita que la página se recargue (`e.preventDefault()`).
- `signInWithEmailAndPassword` valida correo y contraseña contra Authentication.
- Con el UID del usuario, lee su perfil en `usuarios` y revisa el rol.
- ejecutivo → dashboard.html; trabajador → registro.html; cualquier otra cosa → aviso de error.

**Código completo:**

```
import { signInWithEmailAndPassword } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-auth.js";
import { doc, getDoc } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-firestore.js";
import { auth, db } from "./firebase-init.js";

const form = document.getElementById("form-login");
const mensaje = document.getElementById("mensaje");

function mostrarMensaje(texto, tipo) {
  mensaje.textContent = texto;
  mensaje.className = "mensaje " + tipo;
}

form.addEventListener("submit", async (e) => {
```

```
  e.preventDefault();
  mostrarMensaje("", "");
  const correo = document.getElementById("correo").value.trim();
  const contrasena = document.getElementById("contrasena").value;

  try {
    const credencial = await signInWithEmailAndPassword(auth, correo, contrasena);
    const uid = credencial.user.uid;

    const usuarioDoc = await getDoc(doc(db, "usuarios", uid));
    if (!usuarioDoc.exists()) {
      mostrarMensaje("Tu cuenta no tiene un perfil asignado. Contacta al administrador.", "error");
      return;
    }

    const rol = usuarioDoc.data().rol;
    if (rol === "ejecutivo") {
      window.location.href = "dashboard.html";
    } else if (rol === "trabajador") {
      window.location.href = "registro.html";
    } else {
      mostrarMensaje("Rol no reconocido. Contacta al administrador.", "error");
    }
  } catch (error) {
    mostrarMensaje("Correo o contraseña incorrectos.", "error");
  }
});
```

## 7.8 registro.html

**Ruta: registro.html**

La pantalla del trabajador: barra superior con su nombre y el botón Salir, y el formulario de la pieza.

Qué hace por dentro:

- El **folio** va primero porque es el que dispara el autollenado. Debajo está el `<div id="estado-folio">` donde aparece "Pieza encontrada" o "Folio nuevo".
- Datos de la pieza: nombre, cliente, material y cantidad (`type="number"` para que solo acepte números).
- Datos del movimiento: fecha y hora (solo lectura), zona de origen, zona destino y comentarios.
- Los dos `<select>` de zonas vienen vacíos a propósito: registro.js les agrega las opciones, así no hay que escribir 27 `<option>` a mano.
- `required` hace que el navegador no deje enviar sin nombre, folio y zonas.
- Al final carga `js/registro.js` como módulo.

**Código completo:**

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Registro de pieza</title>
<link rel="stylesheet" href="css/styles.css" />
</head>
<body>
  <div class="topbar">
    <h1>Registro de piezas — <span id="nombre-usuario"></span></h1>
    <button id="btn-salir">Salir</button>
  </div>

  <div class="contenedor">
    <div class="card">
      <h2>Nuevo registro</h2>
```

```
      <form id="form-registro">
        <div class="fila-dos">
          <div>
            <label for="folio">Folio</label>
            <input type="text" id="folio" required autocomplete="off" />
          </div>
          <div>
            <label for="fecha-hora">Fecha y hora</label>
            <input type="text" id="fecha-hora" disabled />
          </div>
        </div>
        <div id="estado-folio" class="estado-folio"></div>

        <label for="nombre-pieza">Nombre de la pieza</label>
        <input type="text" id="nombre-pieza" required />

        <label for="cliente">Cliente</label>
        <input type="text" id="cliente" />

        <div class="fila-dos">
          <div>
            <label for="material">Material</label>
            <input type="text" id="material" />
          </div>
          <div>
            <label for="cantidad">Cantidad</label>
            <input type="number" id="cantidad" min="0" step="1" />
          </div>
        </div>

        <div class="fila-dos">
          <div>
            <label for="zona-origen">Zona de origen</label>
            <select id="zona-origen" required></select>
          </div>
          <div>
            <label for="zona-destino">Zona destino</label>
            <select id="zona-destino" required></select>
          </div>
        </div>

        <label for="comentarios">Comentarios</label>
        <textarea id="comentarios"></textarea>

        <button type="submit" class="primario">Registrar</button>
        <div id="mensaje" class="mensaje"></div>
      </form>
    </div>
  </div>

  <script type="module" src="js/registro.js"></script>
 </body>
 </html>
```

## 7.9 registro.js

**Ruta: js/registro.js**

La lógica del formulario del trabajador.

Qué hace por dentro:

- Llena los dos <select> de zonas con las listas de `zonas.js`.
- Actualiza el campo de fecha y hora cada segundo, solo como referencia visual.
- `protegerPagina("trabajador")` revisa la sesión antes de dejar usar el formulario.

- **Autollenado:** cuando el trabajador sale del campo folio (evento `change`), `buscarFolio` lo pasa a mayúsculas y busca `piezas/{folio}` con `getDoc`. Si existe, `llenarDatosPieza` llena los campos y pone la zona de origen igual a la `ultimaZona` de la pieza.
- Si el folio no existe y antes se habían autollenado datos de otro folio, los limpia para no registrar la pieza con datos equivocados.
- El folio no puede llevar `/` porque Firestore no lo acepta en el ID de un documento.
- **Al registrar** usa un `writeBatch` (lote): guarda el movimiento en `registros` y crea/actualiza la ficha en `piezas` (con `merge: true`). El lote garantiza que se guarden los dos o ninguno.
- La hora que se guarda es `serverTimestamp()`: la del servidor de Firebase, no la de la computadora, para que no se pueda alterar.
- Si se guardó bien muestra el aviso verde y limpia el formulario con `form.reset()`.

**Código completo:**

```
import { collection, doc, getDoc, writeBatch, serverTimestamp } from "https://www.gstatic.com/
firebasejs/13.1.0/firebase-firestore.js";
import { db } from "./firebase-init.js";
import { protegerPagina, configurarBotonSalir } from "./auth-guard.js";
import { ZONAS_ORIGEN, ZONAS_DESTINO, llenarSelectZonas } from "./zonas.js";

llenarSelectZonas("zona-origen", ZONAS_ORIGEN);
llenarSelectZonas("zona-destino", ZONAS_DESTINO);

function actualizarRelojFecha() {
  document.getElementById("fecha-hora").value = new Date().toLocaleString("es-MX");
}
actualizarRelojFecha();
setInterval(actualizarRelojFecha, 1000);

const form = document.getElementById("form-registro");
const mensaje = document.getElementById("mensaje");
const campoFolio = document.getElementById("folio");
const estadoFolio = document.getElementById("estado-folio");

const campo = (id) => document.getElementById(id);

function mostrarMensaje(texto, tipo) {
  mensaje.textContent = texto;
  mensaje.className = "mensaje " + tipo;
}

function mostrarEstadoFolio(texto, tipo) {
  estadoFolio.textContent = texto;
  estadoFolio.className = "estado-folio " + (tipo || "");
}

// El folio es el ID del documento en "piezas": se normaliza para que "a12" y " A12 " sean la misma
pieza.
function normalizarFolio(valor) {
  return valor.trim().toUpperCase();
}

// Firestore no acepta "/" en el ID de un documento, ni los IDs "." o "..".
function folioValido(folio) {
  return folio !== "" && !folio.includes("/") && folio !== "." && folio !== "..";
}

function llenarDatosPieza(pieza) {
  campo("nombre-pieza").value = pieza.nombre || "";
  campo("cliente").value = pieza.cliente || "";
  campo("material").value = pieza.material || "";
  campo("cantidad").value = pieza.cantidad ?? "";
  if (pieza.ultimaZona && ZONAS_ORIGEN.includes(pieza.ultimaZona)) {
    campo("zona-origen").value = pieza.ultimaZona;
```

```
  }
}

function limpiarDatosPieza() {
  ["nombre-pieza", "cliente", "material", "cantidad"].forEach(id => { campo(id).value = ""; });
  campo("zona-origen").selectedIndex = 0;
}

// Evita que los datos del folio anterior se queden en el formulario si el nuevo folio no existe.
let datosAutollenados = false;

async function buscarFolio() {
  const folio = normalizarFolio(campoFolio.value);
  campoFolio.value = folio;

  if (folio === "") {
    mostrarEstadoFolio("", "");
    return;
  }
  if (!folioValido(folio)) {
    mostrarEstadoFolio("El folio no puede llevar el carácter \"/\".", "");
    return;
  }

  mostrarEstadoFolio("Buscando folio...", "");
  try {
    const ficha = await getDoc(doc(db, "piezas", folio));
    if (campoFolio.value !== folio) return;

    if (ficha.exists()) {
      llenarDatosPieza(ficha.data());
      datosAutollenados = true;
      mostrarEstadoFolio("Pieza encontrada: se cargaron sus datos. Puedes corregirlos si algo cambió.",
"encontrado");
    } else {
      if (datosAutollenados) limpiarDatosPieza();
      datosAutollenados = false;
      mostrarEstadoFolio("Folio nuevo: se creará la ficha de la pieza al registrar.", "nuevo");
    }
  } catch (error) {
    mostrarEstadoFolio("No se pudo consultar el folio. Llena los datos a mano.", "");
  }
}

campoFolio.addEventListener("change", buscarFolio);

protegerPagina("trabajador").then((usuario) => {
  document.getElementById("nombre-usuario").textContent = usuario.nombre;
  configurarBotonSalir("btn-salir");

  form.addEventListener("submit", async (e) => {
    e.preventDefault();
    mostrarMensaje("", "");

    const folio = normalizarFolio(campoFolio.value);
    if (!folioValido(folio)) {
      mostrarMensaje("El folio no puede llevar el carácter \"/\".", "error");
      return;
    }

    const textoCantidad = campo("cantidad").value;
    const pieza = {
      nombre: campo("nombre-pieza").value.trim(),
      cliente: campo("cliente").value.trim(),
      material: campo("material").value.trim(),
      cantidad: textoCantidad === "" ? null : Number(textoCantidad)
    };
    const zonaOrigen = campo("zona-origen").value;
    const zonaDestino = campo("zona-destino").value;

    // El movimiento y la ficha se guardan juntos: o se guardan los dos, o ninguno.
```

```
    const lote = writeBatch(db);

    lote.set(doc(collection(db, "registros")), {
      folio,
      nombrePieza: pieza.nombre,
      cliente: pieza.cliente,
      material: pieza.material,
      cantidad: pieza.cantidad,
      zonaOrigen,
      zonaDestino,
      comentarios: campo("comentarios").value.trim(),
      nombreTrabajador: usuario.nombre,
      uidTrabajador: usuario.uid,
      creadoEn: serverTimestamp()
    });

    lote.set(doc(db, "piezas", folio), {
      folio,
      ...pieza,
      ultimaZona: zonaDestino,
      actualizadoPor: usuario.nombre,
      actualizadoEn: serverTimestamp()
    }, { merge: true });

    try {
      await lote.commit();
      mostrarMensaje(`Registro del folio ${folio} guardado correctamente.`, "ok");
      form.reset();
      datosAutollenados = false;
      mostrarEstadoFolio("", "");
      actualizarRelojFecha();
    } catch (error) {
      mostrarMensaje("No se pudo guardar el registro. Intenta de nuevo.", "error");
    }
  });
});
```

## 7.10 dashboard.html

**Ruta: dashboard.html**

La pantalla del ejecutivo: una barra de filtros y el espacio donde se dibuja la tabla.

Qué hace por dentro:

- Filtros: folio, zona origen, zona destino, trabajador, cliente, fecha desde y fecha hasta.
- Los `<select>` empiezan con la opción "Todas" (valor vacío = sin filtro); dashboard.js agrega las zonas.
- El `<div id="zona-tabla">` muestra "Cargando..." hasta que llegan los datos.

**Código completo:**

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Dashboard - Registro de Piezas</title>
<link rel="stylesheet" href="css/styles.css" />
</head>
<body>
  <div class="topbar">
    <h1>Dashboard — <span id="nombre-usuario"></span></h1>
    <button id="btn-salir">Salir</button>
  </div>

  <div class="dashboard-contenedor">
    <div class="filtros">
      <div>
```

```
        <label for="f-folio">Folio</label>
        <input type="text" id="f-folio" placeholder="Buscar folio..." />
      </div>
      <div>
        <label for="f-zona-origen">Zona origen</label>
        <select id="f-zona-origen"><option value="">Todas</option></select>
      </div>
      <div>
        <label for="f-zona-destino">Zona destino</label>
        <select id="f-zona-destino"><option value="">Todas</option></select>
      </div>
      <div>
        <label for="f-trabajador">Trabajador</label>
        <input type="text" id="f-trabajador" placeholder="Nombre..." />
      </div>
      <div>
        <label for="f-cliente">Cliente</label>
        <input type="text" id="f-cliente" placeholder="Cliente..." />
      </div>
      <div>
        <label for="f-desde">Desde</label>
        <input type="date" id="f-desde" />
      </div>
      <div>
        <label for="f-hasta">Hasta</label>
        <input type="date" id="f-hasta" />
      </div>
      <div>
        <button id="btn-filtrar">Filtrar</button>
      </div>
      <div>
        <button id="btn-limpiar">Limpiar</button>
      </div>
    </div>

    <div id="zona-tabla">
      <div class="cargando">Cargando registros...</div>
    </div>
  </div>

  <script type="module" src="js/dashboard.js"></script>
</body>
</html>
```

## 7.11 dashboard.js

**Ruta: js/dashboard.js**

La lógica del dashboard.

Qué hace por dentro:

- `protegerPagina("ejecutivo")` deja pasar solo al rol ejecutivo.
- Descarga todos los registros ordenados del más reciente al más antiguo (`orderBy("creadoEn", "desc")`).
- Los filtros se aplican en el navegador sobre esa lista, así que filtrar es instantáneo.
- La tabla muestra también cliente, material y cantidad de cada movimiento.
- Folio, trabajador y cliente buscan coincidencias parciales sin importar mayúsculas; las fechas incluyen el día completo.
- "Limpiar" borra los filtros y vuelve a mostrar todo.
- `escaparHtml` limpia el texto antes de mostrarlo en la tabla, para que si alguien escribe código HTML en comentarios se vea como texto y no se ejecute.

- Los filtros de zona usan las mismas listas de `zonas.js`.

**Código completo:**

```
import { collection, getDocs, query, orderBy } from "https://www.gstatic.com/firebasejs/13.1.0/firebase-
firestore.js";
import { db } from "./firebase-init.js";
import { protegerPagina, configurarBotonSalir } from "./auth-guard.js";
import { ZONAS_ORIGEN, ZONAS_DESTINO, llenarSelectZonas } from "./zonas.js";

llenarSelectZonas("f-zona-origen", ZONAS_ORIGEN);
llenarSelectZonas("f-zona-destino", ZONAS_DESTINO);

let todosLosRegistros = [];

// El texto lo escriben los trabajadores; se escapa para que no se interprete como HTML.
function escaparHtml(valor) {
  const div = document.createElement("div");
  div.textContent = valor ?? "";
  return div.innerHTML;
}

function formatearFecha(timestamp) {
  if (!timestamp) return "—";
  return timestamp.toDate().toLocaleString("es-MX");
}

function renderizarTabla(registros) {
  const contenedor = document.getElementById("zona-tabla");

  if (registros.length === 0) {
    contenedor.innerHTML = '<div class="vacio">No hay registros que coincidan con el filtro.</div>';
    return;
  }

  const filas = registros.map(r => `
    <tr>
      <td>${formatearFecha(r.creadoEn)}</td>
      <td>${escaparHtml(r.folio)}</td>
      <td>${escaparHtml(r.nombrePieza)}</td>
      <td>${escaparHtml(r.cliente)}</td>
      <td>${escaparHtml(r.material)}</td>
      <td>${escaparHtml(r.cantidad)}</td>
      <td>${escaparHtml(r.zonaOrigen)}</td>
      <td>${escaparHtml(r.zonaDestino)}</td>
      <td>${escaparHtml(r.nombreTrabajador)}</td>
      <td>${escaparHtml(r.comentarios)}</td>
    </tr>
  `).join("");

  contenedor.innerHTML = `
    <table>
      <thead>
        <tr>
          <th>Fecha y hora</th>
          <th>Folio</th>
          <th>Pieza</th>
          <th>Cliente</th>
          <th>Material</th>
          <th>Cantidad</th>
          <th>Zona origen</th>
          <th>Zona destino</th>
          <th>Trabajador</th>
          <th>Comentarios</th>
        </tr>
      </thead>
      <tbody>${filas}</tbody>
    </table>
  `;
}
```

```
function aplicarFiltros() {
  const folio = document.getElementById("f-folio").value.trim().toLowerCase();
  const zonaOrigen = document.getElementById("f-zona-origen").value;
  const zonaDestino = document.getElementById("f-zona-destino").value;
  const trabajador = document.getElementById("f-trabajador").value.trim().toLowerCase();
  const cliente = document.getElementById("f-cliente").value.trim().toLowerCase();
  const desde = document.getElementById("f-desde").value;
  const hasta = document.getElementById("f-hasta").value;

  const filtrados = todosLosRegistros.filter(r => {
    if (folio && !(r.folio || "").toLowerCase().includes(folio)) return false;
    if (zonaOrigen && r.zonaOrigen !== zonaOrigen) return false;
    if (zonaDestino && r.zonaDestino !== zonaDestino) return false;
    if (trabajador && !(r.nombreTrabajador || "").toLowerCase().includes(trabajador)) return false;
    if (cliente && !(r.cliente || "").toLowerCase().includes(cliente)) return false;

    if (r.creadoEn) {
      const fechaRegistro = r.creadoEn.toDate();
      if (desde && fechaRegistro < new Date(desde + "T00:00:00")) return false;
      if (hasta && fechaRegistro > new Date(hasta + "T23:59:59")) return false;
    }

    return true;
  });

  renderizarTabla(filtrados);
}

document.getElementById("btn-filtrar").addEventListener("click", aplicarFiltros);
document.getElementById("btn-limpiar").addEventListener("click", () => {
  document.getElementById("f-folio").value = "";
  document.getElementById("f-zona-origen").value = "";
  document.getElementById("f-zona-destino").value = "";
  document.getElementById("f-trabajador").value = "";
  document.getElementById("f-cliente").value = "";
  document.getElementById("f-desde").value = "";
  document.getElementById("f-hasta").value = "";
  renderizarTabla(todosLosRegistros);
});

protegerPagina("ejecutivo").then(async (usuario) => {
  document.getElementById("nombre-usuario").textContent = usuario.nombre;
  configurarBotonSalir("btn-salir");

  try {
    const snapshot = await getDocs(query(collection(db, "registros"), orderBy("creadoEn", "desc")));
    todosLosRegistros = snapshot.docs.map(d => d.data());
    renderizarTabla(todosLosRegistros);
  } catch (error) {
    document.getElementById("zona-tabla").innerHTML = '<div class="vacio">Error al cargar los registros.
</div>';
  }
});
```

## 7.12 firestore.rules

**Ruta: firestore.rules**

Copia de las reglas que pegaste en Firebase en el paso 5. Este archivo no hace nada por sí solo en GitHub; se guarda en el proyecto para tener respaldo y saber qué reglas están activas. Si algún día cambias las reglas, cámbialas en los dos lugares.

**Código completo:**

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function estaAutenticado() {
```

```
      return request.auth != null;
    }

    function rolDeUsuario() {
      return get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data.rol;
    }

    // Solo cada usuario puede leer su propio perfil. Se crean a mano desde la consola.
    match /usuarios/{userId} {
      allow read: if estaAutenticado() && request.auth.uid == userId;
      allow write: if false;
    }

    // Cualquier usuario con sesión puede crear registros (trabajador).
    // Solo el rol "ejecutivo" puede leer/listar todos los registros.
    // Nadie puede editar ni borrar registros ya creados.
    // Ficha de cada pieza, con el folio como ID. Todos los usuarios con sesión la pueden leer
    // (para el autollenado) y crear/actualizar al registrar un movimiento. Nadie la puede borrar.
    match /piezas/{folio} {
      allow read: if estaAutenticado();
      allow create, update: if estaAutenticado() && request.resource.data.folio == folio;
      allow delete: if false;
    }

    match /registros/{registroId} {
      allow create: if estaAutenticado();
      allow read, list: if estaAutenticado() && rolDeUsuario() == 'ejecutivo';
      allow update, delete: if false;
    }
  }
}
```

## 8. Cómo se conecta todo: el <script type="module">

En el paso 3, Firebase te mostró un bloque <script type="module"> con el import, tu firebaseConfig y el initializeApp todo junto. Este proyecto hace lo mismo, pero repartido en archivos:

```
index.html     --carga-->  login.js      --importa-->  firebase-init.js  --importa-->  firebase-config.
js
registro.html  --carga-->  registro.js   --importa-->  firebase-init.js + auth-guard.js + zonas.js
dashboard.html --carga-->  dashboard.js  --importa-->  firebase-init.js + auth-guard.js + zonas.js
```

- Cada HTML tiene al final del <body> una sola etiqueta `<script type="module" src="js/....js"></script>`. Es la misma etiqueta que te mostró Firebase, pero en lugar de tener el código adentro, apunta a un archivo con `src`.
- Cada JS de pantalla hace `import { auth, db } from "./firebase-init.js"` para reusar la conexión, en vez de repetir initializeApp en cada página.
- `type="module"` es obligatorio: es lo que permite usar `import`. Si se quita, el sitio deja de funcionar.
- Va al final del <body> para que el HTML ya exista cuando el JS busque los campos por su id.

## 9. Probar el sitio en tu computadora

Si abres index.html con doble clic, **no va a funcionar**: el navegador bloquea los módulos (type="module") cuando la dirección empieza con `file://`. Por eso se usa Live Server:

1. En VS Code, clic derecho sobre **index.html** → **"Open with Live Server"**.
2. Se abre el navegador en una dirección como `http://127.0.0.1:5500/index.html`.

3. Entra con el usuario trabajador y escribe un folio nuevo (ej. PRUEBA1). Al salir del campo debe decir "Folio nuevo". Llena los datos, registra y revisa en Firestore → Datos que aparecieron las colecciones **registros** y **piezas**.
4. Escribe otra vez el mismo folio: al salir del campo debe decir "Pieza encontrada", llenar los datos solos y poner como zona de origen la zona destino que elegiste antes.
5. Clic en Salir, entra con el usuario ejecutivo y revisa que el registro aparezca en la tabla.
6. Cada vez que guardes un archivo, Live Server recarga la página solo.

Si algo falla, presiona **F12** en el navegador y revisa la pestaña **Consola**: ahí aparece el error exacto (por ejemplo, un nombre de archivo mal escrito o una regla que bloquea).

## 10. Subir el proyecto con GitHub Desktop y activar GitHub Pages

1. Crea el repositorio en **github.com** (botón New), público, ej. "registro-piezas".
2. Abre **GitHub Desktop** e inicia sesión con tu cuenta de GitHub.
3. Clic en **File** → **Clone repository**. En la pestaña "GitHub.com" elige el repositorio, en "Local path" elige dónde guardarlo y clic en **Clone**.
4. Copia **todo el contenido** de tu carpeta del proyecto (index.html, registro.html, dashboard.html, firestore.rules, la carpeta css y la carpeta js) dentro de la carpeta clonada. Los archivos deben quedar en la raíz del repositorio, no dentro de otra subcarpeta.
5. Regresa a GitHub Desktop: los archivos nuevos aparecen en la pestaña **Changes**.
6. Abajo a la izquierda escribe un mensaje como **"Primera versión"** y clic en **"Commit to main"**.
7. Arriba, clic en **"Push origin"** para subir los cambios.
8. En el navegador, entra al repositorio en github.com → **Settings** → **Pages**.
9. En "Source" elige **Deploy from a branch**, rama **main** y carpeta **/ (root)**. Guarda.
10. Espera 1-2 minutos. La dirección del sitio será `https://TU_USUARIO.github.io/registro-piezas/` (en este caso, `https://jaredfg.github.io/registro-piezas/`).

Para cualquier cambio futuro el ciclo es siempre: editar y guardar el archivo → aparece en "Changes" → mensaje → "Commit to main" → "Push origin". GitHub Pages se actualiza solo unos minutos después.

## 11. Alerta "Secrets detected" de GitHub y proteger la API key (opcional)

Al subir el proyecto, GitHub puede mandar un aviso **"Action needed: Secrets detected"** señalando la **Google API Key** en js/firebase-config.js. Es esperado: GitHub detecta cualquier texto que parezca llave de Google, sin distinguir si es de Firebase web.

- La apiKey de Firebase web **no es una contraseña**: está hecha para viajar al navegador, y cualquiera que abra el sitio puede verla. Solo identifica a qué proyecto conectarse.
- Lo que protege los datos son las **reglas de Firestore** (paso 5) y el **login**: sin un usuario válido nadie puede leer ni escribir.

- Lo que **nunca** debe subirse es una clave de cuenta de servicio (un archivo .json con "private_key"). Este proyecto no usa ninguna.
- Puedes cerrar la alerta en GitHub (Security → Secret scanning → la alerta → Close as → "False positive" o "Used in tests").

Para más tranquilidad, restringe la llave a tus dominios (opcional, 5 minutos):

1. Entra a **console.cloud.google.com** con la misma cuenta y elige el proyecto (registropiezas-6039d).
2. Menú → **APIs y servicios** → **Credenciales**. Abre la llave que dice "Browser key (auto created by Firebase)".
3. En "Restricciones de aplicaciones" elige **Sitios web** y agrega:
   `https://jaredfg.github.io/*`
   `https://registropiezas-6039d.firebaseapp.com/*`
   `http://127.0.0.1:5500/*` (para seguir probando con Live Server)
4. Guarda. Los cambios tardan unos minutos en aplicar.

## 12. Probar el sitio publicado

1. Abre la dirección de GitHub Pages y entra con el usuario trabajador.
2. Registra una pieza de prueba: debe salir el aviso verde y el formulario quedar limpio.
3. En Firestore, confirma que apareció el documento en "registros" y la ficha en "piezas".
4. Vuelve a escribir el mismo folio y confirma que se autollenan los datos.
5. Sal y entra con el usuario ejecutivo: debe abrir el dashboard con ese registro.
6. Prueba los filtros (folio, zonas, trabajador, rango de fechas) y el botón Limpiar.
7. Prueba la seguridad: con sesión de trabajador escribe .../dashboard.html en la barra; debe regresarte al formulario.

## 13. Agregar un usuario nuevo más adelante

1. Seguridad → Authentication → Users → Agregar usuario (correo y contraseña).
2. Copiar su UID.
3. Firestore → colección "usuarios" → Agregar documento, con ese UID como ID y los campos nombre y rol.

> No se toca código ni se vuelve a publicar: todo pasa en la consola de Firebase. Para quitarle el acceso a alguien, desactívalo o elimínalo en Authentication → Users.

## 14. Cambios comunes

### Cambiar las zonas (nombres, agregar o quitar)

Todo se hace en un solo archivo: `js/zonas.js`. Cada zona va entre comillas y separada por coma. Por ejemplo, si las zonas reales fueran áreas con nombre:

```
export const ZONAS_ORIGEN = [
```

```
  "Externo",
  "Almacén general", "Línea 1", "Línea 2", "Pintura", "Calidad"
];

export const ZONAS_DESTINO = [
  "Almacén general", "Línea 1", "Línea 2", "Pintura", "Calidad", "Embarques"
];
```

- Para **agregar** una zona, escríbela entre comillas y sepárala de las demás con una coma.
- Para **quitar** una, bórrala junto con su coma.
- Origen y destino pueden ser distintas (en el ejemplo, "Embarques" solo existe como destino).
- Cuida que cada nombre tenga sus dos comillas y que no falten comas entre zonas; si algo queda mal, las zonas dejan de aparecer (F12 → Console muestra el error).
- Guarda, prueba con Live Server y sube con Commit + Push. No hay que tocar Firebase.

> Los registros ya guardados conservan el nombre de zona que tenían. Si renombras una zona (ej. "Zona A" a "Almacén"), los registros viejos seguirán diciendo "Zona A" y el filtro "Almacén" no los va a encontrar.

### Cambiar los colores

Edita las variables de `:root` al inicio de `css/styles.css`.

### Agregar un dato nuevo de la pieza (ej. "lote")

Como es un dato de la pieza, tiene que guardarse en la ficha y autollenarse igual que cliente o material:

1. En **registro.html**, junto a los demás datos de la pieza, agrega: `<label for="lote">Lote</label>` `<input type="text" id="lote" />`
2. En **registro.js**, en cuatro lugares:
   - en el objeto `pieza`: `lote: campo("lote").value.trim(),`
   - en el `lote.set` de registros: `lote: pieza.lote,`
   - en `llenarDatosPieza`: `campo("lote").value = pieza.lote || "";`
   - en `limpiarDatosPieza`, agrega `"lote"` a la lista de ids.
3. En **dashboard.js**, dentro de `renderizarTabla`, agrega un `<th>Lote</th>` en el encabezado y un `<td>${escaparHtml(r.lote)}</td>` en las filas, en la misma posición.
4. Prueba con Live Server y sube con Commit + Push. Las reglas no cambian.

> Los registros y fichas viejos no tendrán ese campo; `escaparHtml` devuelve vacío en ese caso, así que la celda sale en blanco en lugar de "undefined".

### Corregir o revisar la ficha de una pieza

Lo normal es corregirla desde el mismo formulario (al registrar el siguiente movimiento se guarda lo corregido). Si hace falta hacerlo sin registrar un movimiento, entra a Firestore → Datos → colección **piezas** → el documento con ese folio, y edita el campo.

## 15. Siguientes pasos posibles

- Pantalla dentro del dashboard para dar de alta usuarios sin entrar a la consola de Firebase.
- Gráficas (piezas por zona, por día, por trabajador) además de la tabla.

- Exportar los registros filtrados a Excel/CSV.
- Paginación si los registros crecen a miles (hoy el dashboard descarga todos de una vez).